

高级算法课程报告：用算法知识解决在线算法问题

张思成

学号：241250117

2026 年 6 月 16 日

摘要

高级算法课程要求学生能够把复杂度分析、分治、图搜索、贪心、动态规划等知识用于实际问题。本文以 LeetCode 风格题目为对象，重点分析“合并 K 个升序链表”，并结合课程表、跳跃游戏、零钱兑换和滑动窗口最大值等问题说明不同算法思想的适用条件。报告内容包括基础知识、解题思路、核心代码、复杂度证明和过程心得。

关键词：复杂度；分治；堆；对抗策略；平摊分析；图搜索；贪心；动态规划

1 问题背景与建模

在线算法题的难点不只在写出程序，还在于把题目抽象成清晰的算法模型，并证明该模型下的程序在最坏情况下仍然有效。我选择“合并 K 个升序链表”作为主问题。题目给出 k 个已经升序的链表，要求合并成一个新的升序链表。若所有节点总数为 N ，直接做法是每次扫描 k 个链表头部，选出最小节点接入答案。这种方法每处理一个节点要比较 k 次，总时间复杂度为 $O(Nk)$ ，在 k 较大时效率低。

该题的核心结构是：每个链表当前头节点都是本链表尚未合并部分的最小元素，因此全局最小节点必然在这些头节点中。问题就转化为动态集合中反复取得最小元素。课程中学习的堆正好适合维护这种集合，所以可以用最小堆优化。该建模过程说明，算法设计首先要识别数据的有序性和操作需求，而不是直接从样例出发写循环。

在实现前还要明确输入规模对方法选择的约束。若链表数量和节点总数都较大，逐轮扫描会把大量时间浪费在重复比较上；若使用堆或分治，每个节点只在有限次数的结构维护中出现，复杂度随规模增长更平稳。这个判断过程正是高级算法课程强调的“先分析，再实现”。

2 复杂度函数与最坏情况

设输入规模为 n ，算法基本操作次数为 $T(n)$ 。若存在常数 c 和 n_0 ，使得当 $n \geq n_0$ 时有 $T(n) \leq cf(n)$ ，则称 $T(n) = O(f(n))$ 。类似地， $\Omega(f(n))$ 表示下界， $\Theta(f(n))$ 表示

上下界同阶。复杂度函数的意义在于忽略常数差异，关注输入规模趋大时的增长趋势。

最坏复杂度给出任意输入下的性能保证。平均复杂度需要明确输入分布，若题目没有给出概率模型，不能用“通常较快”代替证明。最优性则要和问题下界比较，例如比较排序在最坏情况下需要 $\Omega(n \log n)$ 次比较，因此 $O(n \log n)$ 的归并排序和堆排序在比较模型下具有渐近最优性。在线评测往往包含极端数据，所以最坏复杂度比少量样例运行结果更可靠。

证明复杂度时，需要说明循环次数、递归层数和数据结构操作代价，而不能只给出结论。以链表合并为例，若使用堆，就要分别说明堆大小不超过 k 、每个节点最多入堆和出堆一次、每次堆操作为 $O(\log k)$ 。这些条件合在一起，才能得到 $O(N \log k)$ 。

3 堆与分治法求解链表合并

使用最小堆时，堆中保存每个非空链表的当前头节点。每次弹出堆顶节点，它就是所有未合并节点中的最小值；把它接入结果链表后，如果它还有后继节点，就把后继节点放入堆。堆的大小不超过 k ，一次插入或删除堆顶的代价为 $O(\log k)$ ，总共处理 N 个节点，所以时间复杂度为 $O(N \log k)$ ，额外空间复杂度为 $O(k)$ 。

在示例输入 $[[1, 4, 5], [1, 3, 4], [2, 6]]$ 中，堆会依次取出 $1, 1, 2, 3, 4, 4, 5, 6$ ，输出链表保持升序。该结果说明堆并没有改变节点内容，只改变了选择下一个节点的方式，使每一步都能在较小代价下找到当前最小值。

```
import heapq

def mergeKLists(lists):
    heap = []
    for i, node in enumerate(lists):
        if node:
            heapq.heappush(heap, (node.val, i, node))

    dummy = ListNode(0)
    tail = dummy
    while heap:
        _, i, node = heapq.heappop(heap)
        tail.next = node
        tail = tail.next
        if node.next:
            heapq.heappush(heap, (node.next.val, i, node.next))
    return dummy.next
```

该算法的正确性可以用不变式说明：每一轮开始时，堆中保存每个链表尚未合并部分的最小节点。因此堆顶一定是全局最小的未合并节点，把它加入结果不会破坏升序性。随后把该节点的后继加入堆，又恢复了这个不变式。

同一问题也能用分治法解决。把 k 个链表两两合并，第一层处理所有 N 个节点，第二层仍处理所有 N 个节点，直到只剩一个链表。层数为 $\log k$ ，所以总时间复杂度也

是 $O(N \log k)$ 。递归关系可写为

$$T(k, N) = 2T(k/2, N/2) + O(N).$$

按层展开，每层代价为 $O(N)$ ，共有 $O(\log k)$ 层。堆法适合动态维护最小值，分治法更接近归并排序，两者都比 $O(Nk)$ 的直接扫描更优。

4 图搜索、贪心与动态规划

课程表问题可以建模为有向图。课程是顶点，先修关系是有向边；若存在有向环，就无法完成全部课程。深度优先搜索使用“未访问、正在访问、访问完成”三种状态，若递归中遇到正在访问的节点，就说明存在环。广度优先搜索则统计入度，从入度为零的课程开始做拓扑排序。两种方法都只访问每个顶点和每条边一次，时间复杂度为 $O(V + E)$ 。DFS 更适合判环，BFS 更适合输出学习顺序。

跳跃游戏体现贪心法。遍历数组时维护最远可达位置 *farthest*。若当前下标 $i > \textit{farthest}$ ，说明当前位置无法到达，答案为假；否则更新 $\textit{farthest} = \max(\textit{farthest}, i + \textit{nums}[i])$ 。该方法的贪心思想不是每一步跳得最远，而是在扫描过程中保存所有可达位置的最大扩展能力。其时间复杂度为 $O(n)$ ，空间复杂度为 $O(1)$ 。

零钱兑换体现动态规划。设 $dp[x]$ 为凑出金额 x 的最少硬币数，初始 $dp[0] = 0$ ，状态转移为

$$dp[x] = \min(dp[x], dp[x - \textit{coin}] + 1).$$

若硬币种类数为 m ，目标金额为 *amount*，时间复杂度为 $O(m \cdot \textit{amount})$ 。该题不能随意使用贪心。例如面额 $[1, 3, 4]$ 凑 6，每次选最大面额会得到 $4 + 1 + 1$ ，而最优解是 $3 + 3$ 。这说明局部最优不一定推出全局最优，动态规划更适合具有最优子结构和重叠子问题的场景。

5 平摊分析与对抗策略

滑动窗口最大值要求输出每个长度为 k 的窗口最大值。若每个窗口重新扫描，复杂度为 $O(nk)$ 。单调队列保存下标，并保证对应值从队首到队尾单调不增。新元素入队前，队尾所有不大于它的元素都可删除，因为它们在未来窗口中不会再成为最大值。虽然某次操作可能弹出多个元素，但每个下标最多入队一次、出队一次，所以所有队列操作总数为 $O(n)$ 。这就是平摊分析：单步代价可以不恒定，但整体分摊后每个元素只承担常数操作。

对抗策略强调从最坏输入角度检验算法。若滑动窗口只记录当前最大值，当最大值离开窗口时必须重新扫描。对手可以构造严格递减数组，使每次滑动后最大值都失效，从而让算法退化为 $O(nk)$ 。单调队列通过保留有序候选集合避免这种退化。快速排序也有类似问题：若固定选择第一个元素为主元，已经有序的数组会使划分极不均衡，最坏复杂度达到 $O(n^2)$ 。因此可靠算法必须能抵抗极端输入。

6 过程心得

通过这些题目可以看到，高级算法知识不是互相独立的模板，而是一组分析问题的工具。复杂度函数帮助判断算法是否能承受输入规模；堆和分治降低链表合并成本；DFS 和 BFS 解决图上的可达性与环检测；贪心法依赖可证明的局部选择性质；动态规划依赖状态定义和转移方程；平摊分析解释数据结构整体高效的原因；对抗策略提醒我们不要只相信普通样例。

我在解题中的最大体会是，先建模再编码。只要问题结构、算法不变式和复杂度证明清楚，代码边界条件就更容易处理。一个完整的算法解答应包括解题思路、实现方法、正确性说明和复杂度分析。这样写出的程序不仅能通过示例，也能面对规模约束和最坏输入。

此外，不同方法之间不能只比较代码长短。直接扫描实现最短，但复杂度较高；堆法多维护一个数据结构，却获得更好的最坏情况保证；动态规划代码看似固定，但状态定义错误就会导致整体错误。课程学习让我认识到，算法的价值体现在可证明的效率和正确性上，而不是表面上的写法简洁。

最终报告中的各个例子都围绕同一条主线展开：先判断问题结构，再选择算法工具，最后用复杂度和不变式检验方案是否可靠。这也是我对高级算法实践过程的主要理解。